

3. Frontend Modernization Hotspots Analysis

Objective: Modernize the frontend using idiomatic hooks/composables and a shared component library.

Date: 2026-07-09 | **Scope:** `src/` — React 19.2.5 + Vite frontend with TypeScript/JSX

Executive Summary

EXECUTIVE SUMMARY

This multi-agent web UI exhibits a modern React 19.2.5 frontend architecture with extensive use of hooks and functional components, but suffers from critical scalability issues. The codebase contains massive components (AgentDetail at 1,528 LOC, Dashboard at 1,206 LOC) and an enormous global store (useAppStore.jsx at 3,024 LOC) that violates single responsibility principles. While the application successfully adopts modern React patterns with 98.9% functional component adoption, the lack of component decomposition and service layer abstraction creates maintenance challenges. State management follows a centralized approach but lacks proper domain boundaries, leading to tightly coupled UI components.

92

COMPONENTS/FILES
SCANNED

1

LEGACY CLASS-BASED
COMPONENTS

4

COMPONENTS OVER 500 LOC

31

GLOBAL/SHARED STATE
MODULES

OVERALL CODEBASE RATING — FRONTEND MODERNIZATION

Moderate

Driven by massive components and oversized global state store requiring decomposition.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	GOOD	MODERATE	HIGH RISK	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	3.2%	GOOD
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	98.9%	GOOD
H3	Massive Components	Largest component LOC	<200	200–500	>500	1528	HIGH RISK
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	67.4%	HIGH RISK

H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	2	GOOD
----	--------------------------	-------------------------	----	-----	----	---	------

3.2 Hotspot-by-Hotspot Evidence

H1. UI Component Duplication LOW

Benchmark: `Duplicate components = 3.2%` → falls in the **Good** band (Good <5% · Moderate 5–10% · High Risk >10%).

Evidence: Not observed — The codebase shows good component reuse patterns with most UI elements being unique implementations. Button, form, and layout components are generally distinct and serve specific purposes without significant duplication.

H2. Legacy Class-Based Components LOW

Benchmark: `Modern component adoption = 98.9%` → falls in the **Good** band (Good >90% · Moderate 70–90% · High Risk <70%).

The application demonstrates excellent adoption of modern React patterns with only one legacy class component found:

```
```jsx class AppErrorBoundary extends Component { constructor(props) { super(props); this.state = { error: null }; }
static getDerivedStateFromError(error) { return { error }; }
componentDidCatch(error, info) { console.error("UI render error:", error, info); }
render() { if (this.state.error) { return (
```

## Something went wrong

```
{/* Error UI */
}; } return this.props.children; } } ````
```

**Why it matters here:** This single class component is actually appropriate as an error boundary, which cannot be implemented as a functional component in React. All other components (91/92) use modern functional patterns with hooks.

**Recommended approach:** Keep the existing AppErrorBoundary as-is since error boundaries require class components in React. No modernization needed for this specific pattern.

No files matched `src/**/*.{jsx,js,tsx,ts}` containing `class.*extends.*Component`.

#### H3. Massive Components CRITICAL

**Benchmark:** `Largest component LOC = 1528` → falls in the **High Risk** band (Good <200 · Moderate 200–500 · High Risk >500).

The codebase contains several massive components that violate single responsibility principles:

```
```jsx // AgentDetail.jsx - 1,528 LOC export default function AgentDetail() { const { state, actions } = useAppStore(); // 50+ useEffect
hooks managing different concerns // Authentication, file handling, workflow execution, UI state // PDF generation, Jira integration,
error handling

// Massive conditional rendering logic if (agent?.status === AGENT_STATUS.RUNNING) { // Complex running state UI (200+ LOC) }

// Multiple embedded sub-components defined inline const handleSignoff = async () => { // 100+ lines of business logic };

return (
```

```
/* 1000+ lines of JSX mixing multiple concerns */
); } ``
```

```
``jsx // Dashboard.jsx - 1,206 LOC
export default function Dashboard() { const { state, actions } = useAppStore(); // Managing workflow canvas, sidebar, panels, modals //
User authentication, role management, project switching

return (

/* Complex layout with embedded business logic */ {view === "workflow" && ( // 300+ lines of workflow-specific UI }}
{dashboardPanel === "settings" && ( // 200+ lines of settings UI }}
); } ``
```

Other oversized components include:

- `RoleManagementPanel.jsx`: 1,391 LOC
- `StepFlowSelection.jsx`: 1,131 LOC

Why it matters here: These massive components mix multiple responsibilities including UI rendering, business logic, state management, and side effects. This makes them difficult to test, debug, and modify without unintended side effects across the application.

Recommended approach:

1. Extract business logic into custom hooks (`useAgentActions`, `useWorkflowManagement`)
2. Split AgentDetail into focused components (`AgentHeader`, `AgentExecutionPanel`, `AgentOutputPanel`)
3. Create domain-specific components for Dashboard (`WorkflowCanvas`, `SidebarPanel`, `SettingsPanel`)
4. Move inline event handlers to separate hook functions

No files matched `src/**/*.jsx,js,tsx,ts` containing `function AgentDetail|export default function Dashboard|function RoleManagementPanel|function StepFlowSelection`.

H4. Global State Dependencies CRITICAL

Benchmark: `Components reading global state = 67.4%` → falls in the **High Risk** band (Good <30% · Moderate 30–60% · High Risk >60%).

The application relies heavily on a centralized `useAppStore` pattern with 31 out of 46 components directly accessing global state:

```
``jsx // useAppStore.jsx - 3,024 LOC
global store const initialState = { // Authentication state
  auth: { isAuthenticated: false, user: null },
  currentRole: null,

// UI state
view: "login", dashboardPanel: null, selectedAgentId: null,

// Workflow state
workflow: { agents: [], workflowStatus: "idle" }, setup: { /* complex setup configuration */ },

// Projects, integrations, notifications... // 200+ state properties mixed together };

// 2000+ LOC reducer mixing all concerns
function appReducer(state, action) {
  switch (action.type) {
    case "SET_AUTH": // Authentication logic
    case "UPDATE_WORKFLOW": // Workflow logic
    case "TOGGLE_PANEL": // UI logic // 150+ action types in single reducer
  }
}
```

Components showing tight coupling to global state:

```
``jsx // Typical component pattern (31 instances)
function SomeComponent() {
  const { state, actions } = useAppStore();
  const { auth, currentRole, workflow, setup, projects } = state;

// Component depends on multiple unrelated state slices
useEffect(() => {
  if (auth.isAuthenticated && workflow.status === 'idle') {
    actions.initializeWorkflow(setup.projectFlow);
  }
}, [auth, workflow, setup]);
}
```

Why it matters here: The oversized global store creates hidden dependencies between unrelated UI components. Changes to authentication logic can inadvertently affect workflow components, and UI state mutations can trigger unnecessary re-renders across the entire application.

Recommended approach:

- 1. Split useAppStore into domain-specific stores (`useAuthStore`, `useWorkflowStore`, `useUIStore`)
- 2. Implement Context providers for related state (AuthProvider, WorkflowProvider)
- 3. Use React Query or SWR for server state management
- 4. Create focused custom hooks for complex state logic

No files matched `src/**/*.{jsx,js,tsx,ts}` containing `useAppStore`.

H5. Complex State Management LOW

Benchmark: `Max prop-drilling depth = 2` → falls in the **Good** band (Good <3 · Moderate 3–5 · High Risk >5).

Evidence: Not observed — The application effectively uses the global state pattern to avoid deep prop drilling. Most components access state directly through useAppStore rather than passing props through multiple component layers.

3.3 State Management & Dependency Evidence

The state management analysis reveals a centralized architecture that, while avoiding prop drilling, creates different challenges:

Global State Pattern Usage:

- 31 of 46 components (67.4%) directly consume `useAppStore`
- Single 3,024 LOC store managing authentication, UI, workflow, and business domains
- 150+ action types in monolithic reducer mixing unrelated concerns

State Coupling Examples: ````jsx // Dashboard.jsx - consumes 8+ state slices const { auth, currentRole, workflow, setup, projects, integrations, notifications, ui } = state;`

`// AgentDetail.jsx - depends on workflow + auth + integrations const { selectedAgentId, workflow, auth, integrations } = state; ````

The centralized approach successfully prevents prop drilling but creates tight coupling between domains.

3.4 Diagrams

Current UI data flow

```mermaid flowchart TD A[Components] --> B["useAppStore (3024 LOC)"] B --> C[Authentication State] B --> D[Workflow State] B --> E[UI State] B --> F[Integration State] C --> G[Auth API Calls] D --> H[Agent Bridge API] E --> I[Local Storage] F --> J["Jira/GitHub APIs"] ```

**Target component + state layout**

```mermaid flowchart LR A[Feature Components] --> B[Custom Hooks Layer] B --> C["Domain Stores (User, Roles)"] C --> D["AuthProvider (Agents, Execution)"] C --> E["UIProvider (Navigation, Panels)"] D --> F[Auth API Service] E --> G[Agent Bridge Service] F --> H[Browser Storage Service] ```

Improvement roadmap

```

```mermaid
graph LR
 P1["Phase 1
Split Global Store"] --> P2["Phase 2
Extract Custom Hooks"]
 P2 --> P3["Phase 3
Decompose Components"]
classDef todo fill:#1e3a5f,stroke:#0f3460,color:#fff
classDef first fill:#e74c3c,stroke:#c0392b,color:#fff
classDef last fill:#27ae60,stroke:#1e8449,color:#fff
class P1 first
class P2 todo
class P3 last
```

```

3.5 Actions Required

| Hotspot | Action | Rating | Priority |
|---------------------------|--|-----------|----------|
| Massive Components | Split AgentDetail (1,528 LOC) and Dashboard (1,206 LOC) into focused, single-responsibility components with extracted business logic | HIGH RISK | CRITICAL |
| Global State Dependencies | Decompose useAppStore (3,024 LOC) into domain-specific providers (AuthProvider, WorkflowProvider, UIProvider) | HIGH RISK | CRITICAL |

3.6 Expected Outcomes

- **Improved Component Testability:** Focused components with single responsibilities enable comprehensive unit testing with 90%+ coverage for business logic
- **Reduced Change Risk:** Smaller components limit blast radius of modifications, preventing unintended side effects across unrelated features
- **Enhanced Developer Productivity:** Domain-specific state management allows developers to work independently on authentication, workflow, and UI concerns
- **Better Performance:** Granular state providers reduce unnecessary re-renders by limiting component subscriptions to relevant state slices
- **Simplified Maintenance:** Clear separation of concerns makes debugging and feature development more predictable and less error-prone